TECH CHALLENGE · FASE 4 · RELATÓRIO TÉCNICO (US21)

Monitoramento médico multimodal

Vídeo, áudio, sinais vitais, prescrições e movimentação de pacientes, com detecção de anomalias e alertas para a equipe.

Repositório	github.com/samuelmaxi/monitoramento-medico-multimodal
Fonte	docs/relatorio_tecnico.md
Data	novembro de 2026

Trabalho acadêmico. Não é dispositivo médico e não foi validado clinicamente.

Introdução

Tech Challenge, Fase 4 (US21). Este relatório consolida a arquitetura, os modelos, os resultados e as limitações do sistema. Os detalhes de cada história ficam nos relatórios de rastreabilidade ao DoD em `docs/`, citados em cada seção. Os caminhos são relativos à raiz do repositório.

> **Aviso.** Este sistema **não é um dispositivo médico** e **não foi validado clinicamente**. É um trabalho acadêmico. Os achados indicam pontos para a equipe avaliar. Nenhum deles é diagnóstico nem substitui julgamento clínico. Os limiares e as prioridades de alerta são provisórios e precisam de revisão por profissionais de saúde antes de qualquer uso real.

Sumário

1. Escopo
2. Fluxo multimodal
3. Modelos por tipo de dado
4. Resultados
5. Exemplos de anomalias detectadas
6. Datasets, licenças e LGPD
7. Limitações e trabalhos futuros
8. Referências
9. Apêndice A. Módulo de áudio vocal

1. Escopo

O sistema recebe quatro tipos de dado de pacientes internados ou em atendimento e publica achados num contrato único, que alimenta um motor de alertas para a equipe:

Tipo de dado	O que se detecta	US
Vídeo (fisioterapia, leito, cirurgia)	postura fora da faixa, assimetria, queda, pessoa ou objeto em área crítica	US05 a US08
Áudio de consulta (texto)	transcrição, termos clínicos críticos, sentimento negativo	US11, US13
Áudio de voz (acústica)	fadiga vocal, dificuldade respiratória, possível disartria, desvio da baseline vocal, tosse e respiração audível	US10, US12, US14
Sinais vitais	NEWS2, desvio da baseline do paciente, tendência de piora	US15
Prescrições	mudança abrupta de dose, duplicidade, suspensão abrupta, alta vigilância	US16
Movimentação na internação	imobilidade > 2 h, saída do leito à noite, queda, agitação ou redução de mobilidade	US17
Alertas	prioridade, deduplicação, escalonamento, entrega e auditoria	US19

2. Fluxo multimodal (US04)

Diagrama na Figura 1, ao fim desta seção. Fonte editável: `arquitetura/fluxo_multimodal.mmd` (o GitHub renderiza). Regenerar: `bash scripts/renderizar_diagrama.sh`. Decisões, custos e serviços em `arquitetura/README.md`.

O fluxo tem cinco passos:

1. **Ingestão.** Arquivos em `dados/` e `conteudos/`, fora do Git e baixados por script. Os sinais vitais do MIMIC-IV Demo passam por um replay que entrega as medições em ordem, numa velocidade configurável, para simular o tempo real.
2. **Processamento por modalidade.** Vídeo com OpenCV, OpenPose e YOLOv8. Áudio com `soundfile` e ffmpeg (WAV 16 kHz mono), **Azure AI Speech** (transcrição), **Azure AI Language** (Text Analytics for health e sentimento) e, só local, Praat/parselmouth e YAMNet (biomarcadores vocais e eventos respiratórios). Sinais vitais, prescrições e movimentação em Python local.
3. **Detecção.** Cada detector publica um `EventoAchado` v1 (contrato, JSON Schema em `contratos/schema/evento_achado.v1.schema.json`) no `BarramentoLocal`. O barramento valida cada evento e grava em `saida/eventos.jsonl`.
4. **Fusão** (US18, *late fusion* por paciente). Foi planejada e não está implementada (seção 7).
5. **Alerta.** O motor da US19 assina o barramento com `severity ≥ baixa`, aplica prioridade, deduplicação e escalonamento e envia por e-mail (SMTP) ou webhook (Teams/Slack). Grava log auditável em JSONL ou Azure Cosmos DB.

Os módulos não se chamam entre si; cada um só publica no contrato. Assim cada US é desenvolvida e testada sozinha, e o motor de alertas funciona com qualquer combinação de modalidades presentes. Os serviços Azure ficam só onde o desafio pede IA gerenciada (fala e NLP clínico). Rodam em `brazilsouth`, então áudio e texto clínico não saem do país. O restante roda local, com custo próximo de zero (≈ US$ 0,30 a 2 por rodada completa, com cache obrigatório das respostas Azure em `saida/cache_azure/`).

Cada evento carrega `patient_id` pseudonimizado, `modality`, `event_type`, `timestamp`, `score` (0 a 1), `severity` (`info`, `baixa`, `media`, `alta`) e `evidence`. A `evidence` tem `summary` em português, `features` (valores observados), `thresholds` (limiares aplicados, com a versão) e `artifacts` (quadro, trecho de áudio ou transcrição, por referência). Com isso, quem recebe o alerta vê o valor que disparou a regra e a versão do limiar usado.

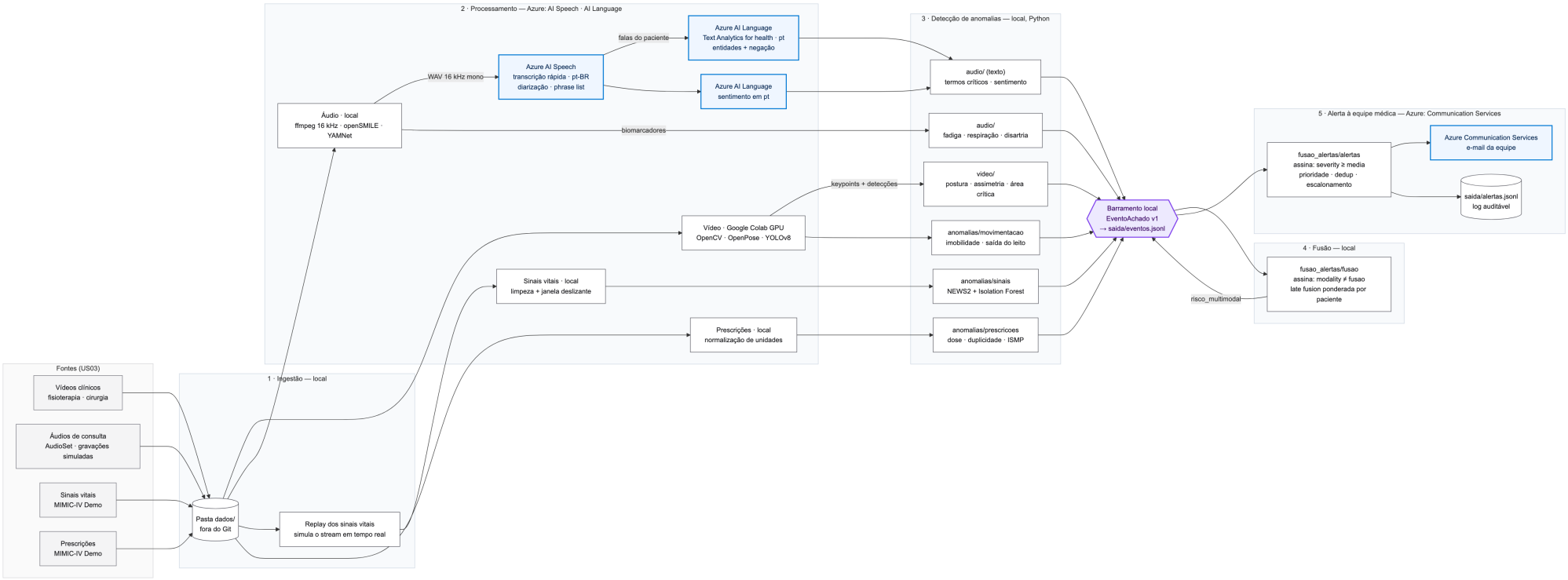


Figura 1. Fluxo multimodal (US04): fontes, processamento, detecção, barramento de eventos, fusão e alerta.

3. Modelos por tipo de dado

3.1 Vídeo

Pré-processamento (US05). OpenCV lê MP4/AVI e amostra a taxa configurada (`VIDEO_TARGET_FPS=5`). Os rostos são anonimizados com o YuNet **antes** de qualquer gravação: tarja sólida, margem de 15% e memória de 3 quadros.

YOLOv8: objetos e áreas críticas (US07). Detalhes no relatório US07.

Item	Valor	Justificativa
Modelo	`yolov8n.pt` (Ultralytics 8.4, pesos COCO)	Leve, roda em CPU e no Colab gratuito. Não houve fine-tuning: o repositório não tem rótulos clínicos
Classes	`person`, `bed`, `chair`, `couch`, `bottle`	Paciente ou cuidador e o mobiliário que define a borda e a lateral do leito
Confiança	`conf = 0,25`	Padrão Ultralytics, comparável à literatura
IoU (NMS e acerto)	`0,5`	Critério PASCAL VOC (mAP@0.5). O mesmo valor na inferência e na avaliação
Rastreamento	ByteTrack (`track_id`)	Entrada e saída pareadas por objeto e área
Áreas críticas	polígonos por câmera (`config/leito_uti_07.json`: `lateral_leito`, `borda_leito`)	Desenhados uma vez em resolução de referência e escalados

OpenPose: análise postural (US06, US08). Detalhes no relatório US08.

Item	Valor	Justificativa
Modelo	OpenPose BODY_25 (Caffe, `cv2.dnn`), `video/pose/config.yaml`	25 keypoints, inclusive pés e `MidHip`, que dão a escala do tronco
Entrada	altura 368 px, 10 fps	Padrão do OpenPose. Em CPU leva ~5× a duração do vídeo
Keypoint válido	confiança ≥ 0,3; esqueleto com ≥ 8 keypoints	Abaixo disso o ângulo fica ruidoso
Pós-processamento	pico 0,05; PAF 0,05; 80% de inliers	Valores do OpenPose original

Regras de desvio da US08 (`config/us08_padroes.yaml`, versão **1.2.0**), com persistência mínima de 1 s por padrão:

Desvio	Regra
Postura e repetição (fisioterapia)	as 3 primeiras repetições, supervisionadas, viram a referência do paciente. As seguintes ficam a ±10° (abdução) ou ±15° joelho / ±20° tronco (agachamento)
Queda	centro do quadril desce ≥ 0,7 comprimento de tronco em ≤ 1 s e fica embaixo (janela 0,5 s)
Mudança brusca	tronco gira ≥ 60° em ≤ 0,5 s e se mantém
Presença indevida	classe proibida numa ROI da US07 por mais que o tolerado (borda do leito: 3 s)

Os perfis de exercício comparam com a referência da própria sessão porque ângulos 2D mudam com a pessoa e com a câmera. Um alvo absoluto de 130° na abdução separava as repetições corretas num paciente e as invertia no seguinte.

3.2 Áudio

Azure AI Speech: transcrição (US11). API de transcrição rápida (síncrona, o áudio vai na requisição, sem armazenamento intermediário), com diarização de até 2 falantes e timestamps por frase e palavra. A *phrase list* `app/frases_medicas.txt` (81 termos clínicos em pt-BR) só é usada se o WER passar de 20%. Custom Speech é o passo seguinte e não foi implementado. Retry em 429/5xx (3 tentativas, respeita `Retry-After`) e cache por SHA-256 do áudio.

Limiares do WER, pela orientação da Microsoft: ≤ 20% aceitável (limite do projeto), 5 a 10% bom, ≥ 30% não aceitável.

Azure AI Language: termos críticos e sentimento (US13). Só as falas do **paciente** são enviadas (a pergunta do médico não gera achado). Detalhes no README.

Item	Valor	Justificativa
Text Analytics for health	`language: "pt"` (o serviço não tem `pt-BR`)	Direto em português, sem tradução que troque termo clínico
Filtro de asserção	descarta `negative`, `negative_possible`, associação `other`, `hypothetical`/`conditional`. `past` rebaixa a severidade um nível	"nego dor no peito" e "meu pai teve infarto" não são achados do paciente
Lista de termos	`audio/text/critical_terms.yaml`, 15 termos versionados com sinônimos e UMLS	Lista fechada, revisável pela equipe
Sentimento	`pt-BR` (com `pt` o serviço assume pt-PT). Sentença negativa se score ≥ 0,7; consulta negativa se ≥ 50% das sentenças	Evento `sentimento_negativo` de severidade baixa, que dá contexto à equipe sem disparar alerta urgente

Ingestão, biomarcadores vocais e classificação (US10, US12, US14). Rodam todos localmente, sem Azure: a transcrição só entra como entrada *opcional*, para separar médico e paciente e contar palavras. Desenho completo no Apêndice A e nos relatórios US10, US12 e US14. Os limiares ficam em `audio/vocal/config.yaml`, cada um marcado `[MDVP]` (normalidade do MDVP, citada na US12), `[PRAAT]` (padrão do Praat) ou `[EXP]` (**experimental**, sem validação clínica).

Etapa	Método	Parâmetros e limiares
Conversão (US10)	`soundfile` (libsndfile), com ffmpeg como plano B; reamostragem polifásica (`scipy`); o WAV gravado é relido e validado	PCM 16 bits, 16 kHz, mono. Erro tipado por arquivo (vazio, corrompido, formato, curto, longo) sem parar o lote
Cópia para features (US10)	normalização de pico e remoção de silêncios longos, **só na cópia**; o original não muda (sha256)	pico alvo 0,95, ganho máximo 5×; silêncios ≥ 1 s (−30 dB relativo ao nível de fala) encurtados para 0,3 s
Biomarcadores (US12)	Praat via parselmouth 0.4.7, por segmento de fala do paciente: F0, jitter local, shimmer local, HNR, intensidade, pausas, taxa de fala	F0 75 a 500 Hz; jitter > 1,04 %, shimmer > 3,81 %, HNR < 20 dB `[MDVP]`; pausas ≥ 0,25 s a 25 dB do nível de fala `[EXP]`
Eventos respiratórios (US12)	YAMNet (AudioSet) em ONNX: tosse, respiração, sibilo, arfar, respiração ofegante	score mínimo 0,30 `[EXP]`; sem o modelo baixado a análise segue sem este critério
Baseline (US12)	z robusto (mediana/MAD) contra os áudios anteriores do mesmo paciente	≥ 3 áudios; $\|z\| \geq 3{,}5$ gera `desvio_baseline_vocal`
Classificação (US14)	regras por classe em YAML, com os critérios satisfeitos explicados no evento	fadiga vocal (2 critérios), dificuldade respiratória (2; **alta com 3**), possível disartria (3); nenhuma → `sem_achados` (normal)

Pausas e taxa de fala são medidas no WAV convertido, **antes** da remoção de silêncio, que apagaria justamente as pausas. Sem transcrição não há taxa de fala, e os critérios que dependem dela ficam

"não avaliáveis" e não contam. A única saída de severidade alta é a dificuldade respiratória com 3 critérios; de propósito, não há regra de áudio em `config/alertas_regras.yaml`, porque uma regra por tipo substituiria essa prioridade. Os limiares MDVP valem para gravação de qualidade clínica: áudio com perdas (Opus do WhatsApp, MP3) os ultrapassa, e o achado leva `audio_comprimido`. Toda saída é um **indicativo para avaliação clínica, não um diagnóstico**.

3.3 Detecção de anomalias

Sinais vitais: NEWS2 + ML não supervisionado (US15). Detalhes no relatório US15 e no desenho.

Camada	Método	Parâmetros e limiares
Limpeza	faixas fisiológicas e validade limitada	SpO2 < 50 e FC > 300 são descartadas; o último valor vale por tempo limitado e depois o NEWS2 sai com `news2_parcial`
Clínica	NEWS2 (RCP 2017, SpO2 escala 1) por parâmetro e total	total 5 a 6 ou 3 num parâmetro → `news2_resposta_urgente` (média); total ≥ 7 → `news2_emergencia` (alta)
ML: mudança súbita	z-score robusto (mediana/MAD) sobre 12 h do próprio paciente	$\|z\|$ > **2,5** em **2** medições seguidas; a baseline congela durante o episódio
ML: tendência	inclinação da regressão linear em 6 h, na escala da baseline	limiar **2,5**

Os limiares de ML saíram de uma busca em grade só nos pacientes de `subject_id` par. Entre os pontos de F1 parecido, foi escolhido o de maior recall, porque perder deterioração custa mais que um falso alarme. A arquitetura citava Isolation Forest; a implementação usa z-score robusto porque não precisa de treino, aprende com cada paciente desde as primeiras horas, é explicável (o z vai na evidência) e cumpriu a meta. Tudo roda local, sem o Azure AI Anomaly Detector, que foi descontinuado.

Prescrições: regras + lista ISMP (US16). Detalhes no relatório US16; limiares em `config/us16_prescricoes.yaml` v1.0.0. As unidades são normalizadas antes de qualquer comparação (mcg/g → mg, L → mL).

Regra	Dispara quando
`mudanca_abrupta_dose`	dose diária do mesmo produto e via muda ≥ **50%** em relação às **24 h** anteriores
`duplicidade_terapeutica`	mesmo princípio ativo (também dentro de combinações), via sistêmica, sobreposição ≥ **6 h**
`suspensao_abrupta`	classe que exige desmame (betabloqueador, corticoide, opioide etc.) em uso ≥ **72 h** some sem retorno em **24 h**, com o paciente internado
`medicamento_alta_vigilancia`	primeira ordem na internação de uma das **19 classes** da lista ISMP 2024 / ISMP Brasil 2019

As exceções vieram dos dados: PRN, ordens tituladas, esquema basal + correção de insulina, troca de prescrição (`Inactive (Due to a change order)`), desmame prévio ≥ 25%, suspensão perto da alta. Insulina, anticoagulante e opioide na introdução ficam em severidade baixa: são 55% das introduções e, em média, gerariam 5,7 alertas por internação (fadiga de alarme).

Movimentação: NPIAP/EPUAP + baseline (US17). Detalhes no relatório US17; limiares em `config/us17_movimentacao.yaml` v1.3.0. Duas fontes geram a mesma série: vídeo (pose, áreas da US07, caixa do YOLOv8) e registro de enfermagem do MIMIC-IV (`Position`, `Turn`, `Activity`).

Regra	Limiar
`imobilidade_prolongada`	> **120 min** sem mudança de posição (NPIAP/EPUAP), média; alta a partir de 240 min. Lacuna > 150 min não conta
`saida_do_leito`	alta à noite (22:00 a 06:00), média de dia
`queda`	sempre alta. Pelo vídeo: centro da caixa `person` desce ≥ **0,065** da altura do quadro em ≤ 1,5 s e fica embaixo 0,5 s
`agitacao_subita` / `reducao_mobilidade`	taxa de 6 h ≥ 2× ou ≤ 0,5× a baseline de 24 h do paciente (mínimo 12 h e 3 eventos de histórico)

3.4 Motor de alertas (US19)

Relatório US19, `config/alertas_regras.yaml`. O motor conhece só o contrato, então aceita qualquer `event_type` do catálogo sem mudar código.

- **Prioridade em 3 níveis** (alta, média, baixa), na linha da IEC 60601-1-8. `info` nunca alerta.
- **Fadiga de alarme:** deduplicação por `paciente | modalidade | event_type` em janela de 5 min (alta), 15 min (média) ou 60 min (baixa). Escalonamento por persistência (contagem ou tempo).
- **Entrega:** SMTP e webhook, com outbox (`pending` gravado antes do envio), retentativa exponencial (5, 10, 20 s) e recuperação ao reiniciar. Reconhecimento por link com token HMAC.
- **Registro:** `saida/alertas.jsonl` + trilha append-only, ou Cosmos DB. Painel em `GET /alerts/dashboard`.

4. Resultados

Todos os números foram medidos e podem ser reproduzidos pelos scripts citados. Nas avaliações com dados rotulados, os limiares foram ajustados numa partição e o número reportado vem de outra. **A exceção é a US14** (voz sintética com alterações injetadas, sem partição): os limiares são anteriores à avaliação, mas partem dos mesmos autores dos casos, então o resultado é circular, como a própria linha da tabela registra.

US	Métrica	Meta	Obtido	Atingida?	Observação
US07	mAP@0.5 (COCO val2017, `yolov8n`)	sem meta (braço sem fine-tuning)	**0,528**	n/a	`saida/video/metricas_coco_val2017.json`
US07	mAP@0.5:0.95 · Precision · Recall	n/a	**0,366** · **0,637** · **0,487**	n/a	Confere com o model card (0,373)
US07	AP@0.5 das classes de interesse	n/a	person 0,503 · bed 0,425 · couch 0,431 · bottle 0,296 · chair 0,253	n/a	
US07	mAP@0.5 por classe médica após fine-tuning	≥ 0,50	não medido	✘	Sem rótulos clínicos no repositório. O gate (`scripts/dataset_us07.py treinar`) está pronto
US08	Precision por evento (teste)	≥ 0,70	**0,71**	✔	35 eventos, `scripts/avaliar_us08.py`
US08	Recall por evento (teste)	≥ 0,80	**0,52**	✘	Queda só por pose tem recall 0,23 no FallVision. Abdução e agachamento ficam em 0,64

US	Métrica	Meta	Obtido	Atingida?	Observação
US11	WER (Azure Speech)	≤ 20%	não medido	✘	Rodada de 06/11/2026: os 10 áudios do Eka voltaram HTTP 401. A assinatura Azure do grupo está desativada (seção abaixo). Reproduzir: `python scripts/avaliar_wer.py --baixar 10`
US13	Precision / Recall / F1 de termos críticos	sem meta fixa	não medido	n/a	Mesma assinatura desativada; `AZURE_LANGUAGE_KEY` também está vazia no `.env`. Reproduzir: `python scripts/eval_us13.py`
US10	Conversão para WAV PCM 16 kHz mono; arquivo ruim não para o lote; original preservado	todos	atendido: cada conversão é relida e validada; erro por arquivo em log e em `metadados.csv`; sha256 do original inalterado	✔	`tests/audio/test_ingestao_*.py`
US12	Biomarcadores por segmento do paciente (F0, jitter, shimmer, HNR, intensidade, pausas, taxa de fala) e eventos respiratórios (YAMNet)	todos	implementados. Em voz sintética: F0 148,8 Hz contra 148,35 Hz esperados (0,3 %); jitter injetado de 2 % medido em 1,96 %	✔	YAMNet verificado só em silêncio e em tons, **não** em tosse ou respiração reais
US14	Recall por classe (casos injetados, 60 por classe)	≥ 0,80	fadiga **1,00** · dificuldade respiratória **1,00** · disartria **1,00** (IC95 0,94 a 1,00)	✔	**Circular**: limiares e casos injetados são dos mesmos autores; mede a lógica das regras, não a validade clínica. FP em voz normal sintética 0/60. `saida/audio/avaliacao_us14.json`
US14	Exemplo de alteração vocal detectada	≥ 1	1 caso **sintético** (seção 5.6); os 3 áudios reais (OGG/Opus) não dispararam classe	✔ (sintético)	Os 3 reais ultrapassaram jitter, shimmer e HNR juntos: artefato de codec, não alteração
US15	Recall (teste, 50 pacientes, 2.924 slots)	≥ 0,80	**0,838**	✔	VP 1.264 · FN 244 · FP 574 · VN 842
US15	Precision (teste)	n/a	**0,688**	n/a	Piso: parte dos FP é piora real sem rótulo
US15	F1 (teste)	≥ 0,70	**0,756**	✔	`saida/sinais/relatorio_us15.json`
US15	Latência por medição	tempo real	~0,1 ms	✔	Janela deslizante em `deque`
US16	Recall (casos injetados, modo realista)	≥ 0,80	**0,974** (IC95 0,967 a 0,980)	✔	Cada tipo ≥ 0,94. FP em controles 1,5%. `saida/prescricoes/avaliacao_us16.json`
US16	Recall (modo controlado)	≥ 0,80	**1,000**	✔	FP 0,4%
US17	Recall de queda em vídeo real (teste)	≥ 0,80	**0,875** (14/16, IC95 0,64 a 0,97)	✔ (pontual)	O limite inferior do IC fica abaixo de 0,80. Alarme falso 1/5
US17	Recall nos casos injetados no MIMIC	n/a	queda 1,00 · saída noturna 1,00 · imobilidade 1,00 · agitação 0,988 · redução 0,822	n/a	Verifica a lógica, não a percepção
US19	Latência ingestão → envio, p95, canal local	≤ 10 s	**0,004 s** (p50 0,002 · máx 0,026)	✔	100 achados, webhook HTTP local: mede só o overhead do motor
US19	Latência ingestão → envio, p95, **e-mail SMTP real**	≤ 10 s	**54,97 s** (p50 34,99 · máx 54,97)	✘	10 achados simultâneos, 9 enviados. `saida/latencia_us19_smtp.json`

Metas não atingidas e por quê

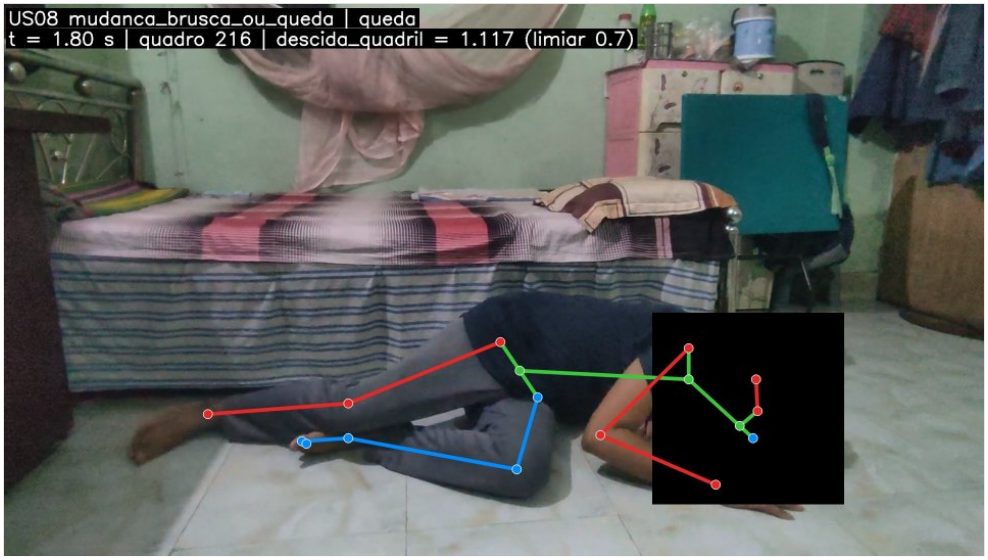
- **US07, fine-tuning em classes médicas.** Não há dataset clínico anotado com licença compatível. O pipeline de treino e o gate por classe existem, mas sem rótulos não há número para declarar.
- **US08, recall de 0,52.** O ponto fraco é a queda vista só pela pose: nos clipes curtos do FallVision a pessoa aparece em menos de 50% dos quadros, ou o clipe acaba logo após o impacto. Na abdução, uma referência contaminada (PM_039 começa com repetição incorreta) derruba o recall. A US17 mostrou a correção: com a descida da caixa do YOLOv8, o recall de queda vai a 0,875.
- **US11 e US13, não medidos.** Na rodada de 06/11/2026 com o `.env` do grupo, o Speech respondeu HTTP 401 aos 10 áudios, em `eastus` e em `brazilsouth`. O Cosmos DB, na mesma assinatura, respondeu `Forbidden: Subscription owning the database account is disabled`. A assinatura Azure está desativada (o crédito do Azure for Students para os recursos quando acaba). A US13 também não tem `AZURE_LANGUAGE_KEY`. Os scripts estão prontos e testados com a API simulada (`pytest audio/tests`). Reativada a assinatura, basta rodá-los e copiar os números. Não há dataset de referência em pt-BR, por isso o WER usa o Eka Medical ASR em inglês indiano. Esse desvio do DoD fica registrado.
- **US19, latência com e-mail real (p95 55 s).** O motor leva 4 ms. O tempo vem da entrega: o `Despachante` (`alertas/despachante.py`) envia um alerta por vez, e o `NotificadorEmail` abre uma conexão SMTP nova a cada e-mail (STARTTLS + login, ~5,5 s no provedor). Com 10 alertas simultâneos, o décimo espera os nove anteriores, e a latência cresce em linha reta (p50 35 s). Um alerta isolado sai em ~5 s, dentro da meta, mas uma rajada não. Um dos 10 e-mails falhou na primeira tentativa e ficou para retentativa. Correção proposta: reutilizar a conexão SMTP e enviar em paralelo por canal (seção 7).
- **US12 e US14, sem validação com voz real.** O recall de 1,00 vem de voz sintética e é circular. Faltam gravações rotuladas por fonoaudiólogo, que o projeto não tem, para validar os limiares `[EXP]` e para medir o YAMNet em tosse e respiração reais. Nenhum dos 3 áudios reais disparou uma classe, então o exemplo documentado é sintético e **falta uma gravação real alterada** para o vídeo. O caminho com transcrição real (`--transcrever`) também não foi exercitado, pela mesma assinatura Azure desativada. Detalhes no Apêndice A.12.

5. Exemplos de anomalias detectadas

Um exemplo por tipo, com o evento resumido e a evidência. IDs de paciente são pseudônimos HMAC.

5.1 Vídeo: queda (FallVision `B_N_03`, perfil `internacao_leito`)

O quadril desceu 1,1 comprimento de tronco em 0,5 s e ficou embaixo. Quadro 216 (1,80 s), anonimizado:



Queda em B_N_03. Quadro de evidência anonimizado.


```
{"modality": "video", "event_type": "mudanca_brusca_ou_queda", "severity": "alta", "score": 0.7087,
 "evidence": {"summary": "Possível queda aos 0,9 s: o quadril desceu 1,1 comprimentos de tronco em 0,5
s e ficou embaixo por 1,1 s.",
   "features": {"regra": "queda", "valor": 1.117, "frame_index": 216},
   "thresholds": {"versao_limiares": "1.1.0", "limiar": 0.7, "janela_minima_s": 0.5}}}
```


Outro desvio real (repetição incorreta de abdução, REHAB24-6 `PM_001`, ombro em 122° contra 135° da referência): `evidencias/us08/rehab24_PM_001_repeticao8_81.80s.jpg`. Entrada e saída de área crítica (US07, `B_D_0016`, 10 eventos pareados): `saida/video/eventos_us07.jsonl` e `saida/video/rois_leito_uti_07.jpg`.

5.2 Áudio: termo crítico afirmado e negação descartada (US13)

Consulta simulada `data/text_samples/consulta-01.json`, anotada em `data/text_samples/annotations.csv`. O paciente diz "estou com uma dor no peito muito forte". O médico pergunta "você sente falta de ar?" e o paciente nega febre. Saída esperada: um `termo_critico` para dor torácica e nenhum achado para a pergunta do médico nem para a febre negada.


```
{"modality": "texto", "event_type": "termo_critico", "severity": "alta",
 "evidence": {"summary": "Paciente relata dor torácica: “dor no peito” (afirmado, sem negação)",
   "features": {"termo": "dor torácica", "nome_normalizado": "Chest Pain", "certeza": "positive",
                "associacao": "subject", "falante": "paciente", "offset_ms": 6512},
   "artifacts": [{"kind": "transcricao", "uri": "data/text_samples/consulta-01.json", "start_ms":
3615, "end_ms": 11675}]}}
```

> Este evento mostra o formato com valores ilustrativos (o score do serviço não está nele). A rodada real com o Azure AI Language grava o exemplo medido em `outputs/text_analysis/eval.md`, que deve substituir este bloco.

5.3 Sinais vitais: NEWS2 de emergência (MIMIC-IV Demo, replay)

Gerado por `python scripts/rodar_us15.py --velocidade 0 --pacientes 3` (o replay reescreve o horário para o instante da execução):


```
{"modality": "sinais_vitais", "event_type": "news2_emergencia", "severity": "alta",
 "patient_id": "pt_b00c3ff10002574317845f45",
 "evidence": {"summary": "NEWS2 = 8 (fr 24, pas 84, fc 91, o2_suplementar 1): emergência",
   "features": {"fr_irpm": 24, "pas_mmhg": 84, "fc_bpm": 91, "spo2_pct": 98, "temp_c": 37.06,
"o2_suplementar": 1,
                "news2_fr": 2, "news2_pas": 3, "news2_fc": 1, "news2_o2_suplementar": 2,
"news2_total": 8,
                "news2_parcial": true, "news2_ausentes": "alerta"},
   "thresholds": {"news2_urgente_total": 5, "news2_emergencia_total": 7, "news2_parametro_unico": 3}}}
```


Na mesma rodada, a camada de ML marcou `desvio_baseline` noutro paciente: "pas 89 desvia da baseline do paciente (mediana 105, z = -2.7)", limiar |z| 2,5 em 2 medições.

5.4 Prescrições: mudança abrupta de anticoagulante (MIMIC-IV Demo)

`pt_472b69fdffa5894af2fbb0c0`, ordens 41490219 → 58986734, `mudanca_abrupta_dose`, média, score 1,0:

> Dose de Enoxaparin Sodium (SC) subiu 400% (40 → 200 mg/dia), 1 h depois do fim da anterior. Antes: 40 mg QPM (40 mg/dia); depois: 100 mg Q12H (200 mg/dia). Limiar: ±50% em 24 h.

Profilaxia trocada por dose plena. Pode ser conduta correta (trombose diagnosticada), mas anticoagulante cinco vezes maior de uma hora para outra pede dupla checagem de peso e função renal. Outros três casos reais, um de cada tipo, no relatório US16.

5.5 Movimentação: imobilidade prolongada (MIMIC-IV Demo)

Em `saida/movimentacao/mimic_eventos.jsonl` (861 eventos válidos de 139 estadias):


```
{"modality": "movimentacao", "event_type": "imobilidade_prolongada", "severity": "alta", "score": 1.0,
 "patient_id": "pt_c2e6536fb12486a755b270c0",
 "evidence": {"summary": "4 h 00 min sem mudança de posição no leito",
   "features": {"minutos_sem_mudanca": 240.0, "reposicionamentos_por_hora": 0.426},
   "thresholds": {"limiar_min": 120.0, "lacuna_max_min": 150.0, "severidade_acima_de_min": "240:alta",
"versao_limiares": "1.3.0"}}}
```


Queda em vídeo real pela descida da caixa: `python -m movimentacao.run video --video conteudos/videos/B_D_0002.mp4` gera um evento `queda` de severidade alta.

5.6 Áudio vocal: dificuldade respiratória (caso sintético)

Voz sintética de dificuldade respiratória (3 frases curtas, pausas longas, voz soprosa), pela CLI `python -m audio.vocal.run`, com o Praat e o YAMNet reais. Nenhum evento respiratório passou do score mínimo; o achado veio de **3 de 4 critérios** (pausas, fala lenta e HNR baixo):


```
{"modality": "audio", "event_type": "dificuldade_respiratoria", "severity": "alta", "score": 0.75,
 "model_version": "audio-achados@0.1.0",
 "evidence": {"summary": "Indicativo para avaliação clínica, não é diagnóstico: padrão compatível com
dificuldade respiratória (3 de 4 critérios): pausas ocupam 50% da fala (limite 40%); taxa de fala 0.92
palavras/s (mínimo 1.8 palavras/s); HNR 18.17 dB (mínimo 20 dB)",
   "features": {"regras_acionadas": "pausas_altas, fala_lenta, hnr_baixo", "proporcao_pausas": 0.499,
                "taxa_fala_pps": 0.919, "hnr_db": 18.17, "jitter_local_pct": 0.519,
                "shimmer_local_pct": 2.635, "eventos_respiratorios": 0},
   "thresholds": {"versao_limiares": "1.0.0", "minimo_criterios": 2, "criterios_forte": 3,
                  "proporcao_pausas_max": 0.4, "taxa_fala_pps_min": 1.8, "hnr_db_min": 20.0}}}
```


O motor de alertas gerou um alerta de prioridade `alta` (origem `padrao:alta`) com a ação sugerida "Avaliar o paciente agora, conforme o protocolo do setor." O envio ficou `failed` só porque nenhum canal (SMTP ou webhook) estava configurado no ambiente; o alerta está registrado no painel. Na mesma rodada dispararam também `fadiga_vocal` e `possivel_disartria` (média): as classes se sobrepõem. Os três áudios reais de voz (OGG/Opus) saíram `normal` com `audio_comprimido`; ver o Apêndice A.13.

6. Datasets, licenças e LGPD

Datasets

Nenhum dado bruto é versionado. Tudo é baixado por script para `dados/`, `data/` ou `conteudos/`, que estão no `.gitignore`. Ficam no Git só os resultados agregados, as anotações próprias e as evidências anonimizadas. Áudio de voz nunca é versionado: o `.gitignore` cobre as extensões de áudio e um teste confere `git ls-files`.

Dataset	Uso	Licença
MIMIC-IV Clinical Database Demo 2.2 (PhysioNet)	sinais vitais (US15), prescrições (US16), enfermagem (US17)	ODbL v1.0
REHAB24-6 (Černek et al., 2024)	fisioterapia, pose e desvios (US06, US08)	CC BY-NC 4.0, uso não comercial com atribuição
FallVision (clipes de leito)	queda (US08, US17)	conferir no repositório de origem antes de redistribuir. Os clipes não são versionados
COCO val2017	métricas do YOLOv8 (US07)	anotações CC BY 4.0; imagens sob os termos do Flickr
Eka Medical ASR Evaluation	WER (US11)	MIT
PriMock57 (Babylon Health, 2022)	transcrições de consultas simuladas (US13)	CC BY 4.0
MedPT (AKCIT, 2026)	perguntas reais de pacientes em pt-BR (US13)	card sem licença: o texto fica fora do Git, só IDs e rótulos são versionados
`data/text_samples/` (15 consultas)	NLP em pt-BR (US13)	criadas pelo grupo, simuladas

Dataset	Uso	Licença
Pesos `yolov8n.pt` e OpenPose BODY_25	inferência	AGPL-3.0 (Ultralytics); OpenPose: licença acadêmica não comercial da CMU
Voz sintética (`audio/vocal/injecao.py`)	testes e avaliação da US14	gerada pelo grupo, em memória; nada a redistribuir
3 áudios de voz (OGG/Opus), só na máquina local	rodada com áudio real (US12)	não versionados; o consentimento do titular precisa ser confirmado
YAMNet em ONNX (espelho `andrelgomes/yamnet-onnx`) e mapa de classes do AudioSet	eventos respiratórios (US12)	Apache-2.0, conforme o script de download, que fixa o SHA-256. O ONNX é uma conversão de terceiros
Praat, via `praat-parselmouth` 0.4.7	biomarcadores vocais (US12)	GPL-3.0 ou posterior: avaliar antes de qualquer redistribuição

Tratamento LGPD

Dados de saúde são dados pessoais sensíveis (LGPD, art. 5º, II). As medidas:

Medida	Como
Pseudonimização na borda (art. 13 §4º)	`patient_id` e `encounter_id` viram `pt_<24 hex>` por HMAC-SHA256 com `PSEUDONYM_KEY` (≥ 32 bytes, só no `.env`), em `contratos/pseudonimizacao.py`. É determinístico para a fusão cruzar modalidades e irreversível sem a chave. Nenhum `subject_id` cru aparece no evento (testado)
Anonimização de imagem	rostos tarjados antes de qualquer gravação (YuNet). Nos quadros de evidência, a cabeça é tarjada pelos keypoints do rosto
Voz (US10, US12, US14)	`patient_id` pseudonimizado antes de qualquer cálculo; a baseline vocal é indexada só pelo pseudônimo e a transcrição é gravada com ele. Praat, YAMNet e regras rodam localmente; só a transcrição opcional envia áudio ao Azure. Metadados com `origem` e `licenca` e o nome do arquivo, nunca o caminho absoluto. `audio_id` validado, porque vira nome de pasta
Minimização	só as falas do paciente vão ao NLP. Evidência por referência: o evento guarda o caminho, não o conteúdo. Alertas não trazem nome
Localização dos dados (art. 33)	Azure Speech e Language em `brazilsouth`, então áudio e texto não saem do país. A transcrição rápida não guarda o áudio em armazenamento na nuvem
Segredos	sem chave no Git. A arquitetura alvo usa `DefaultAzureCredential` com função RBAC mínima (`azure-funcao-minima.json`); em produção, a chave HMAC iria para o Key Vault
Base dos dados	só dados públicos e desidentificados na origem (MIMIC) ou simulados. MIMIC desloca as datas para o futuro
Auditoria	barramento e alertas em JSONL append-only, com reconhecimento por token HMAC

Os três áudios de voz da rodada com áudio real ficam só na máquina local e não são versionados; o consentimento dos titulares precisa ser confirmado. Fora isso, não houve coleta de dados de pessoas reais pelo grupo. Um uso clínico real exigiria base legal (art. 11), relatório de impacto (RIPD), contrato de operador com a Microsoft e política de retenção, e o grupo não fez nenhum deles.

7. Limitações e trabalhos futuros

Não é dispositivo médico validado clinicamente. Nenhum limiar, prioridade ou regra foi revisado por equipe clínica nem avaliado prospectivamente. Uso assistencial exigiria validação clínica e enquadramento regulatório (ANVISA, RDC 657/2022, software como dispositivo médico).

Limitações

- **Avaliação com rótulo sintético ou pequeno.** US15, US16 e US17 (MIMIC) medem anomalias injetadas, não eventos clínicos reais. US17 tem 16 quedas no teste, com rótulos que ainda pedem conferência humana.
- **Vídeo.** Ângulos 2D dependem da câmera. A referência de fisioterapia só vale se as 3 primeiras repetições forem corretas. Câmera vista de cima não mostra queda como descida. A US07 não separa paciente de profissional. Sem fine-tuning, não há classes cirúrgicas. Rostos pequenos, inclinados ou cobertos podem escapar da anonimização.
- **Áudio.** WER e US13 sem rodada real nesta entrega. Não há dataset de referência em pt-BR. O Text Analytics for health é mais maduro em inglês. A API GA não devolve temporalidade, então o passado é inferido por marcadores locais.
- **Áudio vocal (US10, US12, US14).** Os limiares `[EXP]` não têm validação clínica, e a avaliação em voz sintética é circular. Os limiares MDVP são de gravação de qualidade clínica: áudio com perdas (Opus do WhatsApp) ultrapassa os três de uma vez, e o sistema sinaliza `audio_comprimido`, mas não corrige. O módulo não faz supressão de ruído, e ruído de fundo reduz o HNR. O YAMNet não foi medido em tosse ou respiração reais. Sem transcrição não há taxa de fala. Ver as limitações no Apêndice A.15.
- **Prescrições.** O MIMIC registra ordens, não administrações. Sem peso nem indicação, toda mudança grande de dose é anomalia, mesmo quando é a conduta certa. O princípio ativo vem de vocabulário próprio, não do RxNorm.
- **Tempo real.** Só os sinais vitais são streaming (replay). Vídeo e áudio são processados por arquivo.
- **Alertas.** Com SMTP real, a entrega em série estoura a meta de 10 s em rajadas. Teams e Slack reais não foram testados (sem `ALERTA_WEBHOOK_URL`). O armazenamento no Cosmos DB não foi validado contra a conta real, por causa da assinatura desativada. Azure Communication Services Email não foi implementado.
- **Dependência da assinatura Azure.** Transcrição, NLP clínico e Cosmos param juntos quando a assinatura é desativada. Vídeo, sinais vitais, prescrições, movimentação e alertas por SMTP continuam funcionando, porque rodam local.

Trabalhos futuros

1. **Fusão multimodal (US18):** *late fusion* ponderada por paciente e janela, publicando `risco_multimodal` com `related_event_ids`.
2. **Calibrar e validar os biomarcadores vocais (US12, US14):** limiares `[EXP]` com gravações rotuladas por fonoaudiólogo, YAMNet medido em tosse e respiração reais, estimativa de SNR e supressão de ruído, e openSMILE (eGeMAPS) para comparar com o Praat.
3. **Rodar WER e US13 com os recursos Azure** e, se o WER passar de 20%, Custom Speech em pt-BR (treino em `eastus`, cópia para `brazilsouth`).
4. **Fine-tuning do YOLOv8** em classes médicas (instrumentos, monitores) com o gate por classe ≥ 0,50 já implementado.
5. **Calibrar postura pela pose** em vídeo real de leito e confirmar pelo fisioterapeuta a série de referência.

6. **Produção na Azure:** Event Hubs/IoT Hub na ingestão, Service Bus no lugar do barramento (sessões por `patient_id`), Functions ou Container Apps, Key Vault, Application Insights e Health Data Services (FHIR). Ver arquitetura.
7. **Entrega de alertas em paralelo:** conexão SMTP reutilizada (ou ACS Email, que é HTTP) e envio concorrente por canal no `Despachante`, para p95 ≤ 10 s também em rajadas.
8. **Validação clínica** dos limiares e prioridades com a equipe, e estudo retrospectivo com eventos reais rotulados.

8. Referências

1. Royal College of Physicians. *National Early Warning Score (NEWS) 2: Standardising the assessment of acute-illness severity in the NHS.* London: RCP, 2017.
2. Microsoft. *Test accuracy of a Custom Speech model* (Word Error Rate). Microsoft Learn, Azure AI Speech. https://learn.microsoft.com/azure/ai-services/speech-service/how-to-custom-speech-evaluate-data
3. IEC 60601-1-8:2006+A1:2012+A2:2020. *Medical electrical equipment, Part 1-8: General requirements for basic safety and essential performance. Collateral standard: General requirements, tests and guidance for alarm systems in medical electrical equipment and medical electrical systems.*
4. Institute for Safe Medication Practices. *ISMP List of High-Alert Medications in Acute Care Settings*, 2024. ISMP Brasil. *Medicamentos potencialmente perigosos de uso hospitalar: lista atualizada 2019.* Boletim ISMP Brasil, v. 8, n. 1, 2019.
5. European Pressure Ulcer Advisory Panel, National Pressure Injury Advisory Panel, Pan Pacific Pressure Injury Alliance. *Prevention and Treatment of Pressure Ulcers/Injuries: Clinical Practice Guideline.* 3rd ed. EPUAP/NPIAP/PPPIA, 2019.
6. Cao, Z.; Hidalgo, G.; Simon, T.; Wei, S.-E.; Sheikh, Y. OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields. *IEEE TPAMI*, 43(1), 2021.
7. Jocher, G.; Chaurasia, A.; Qiu, J. *Ultralytics YOLOv8*, 2023. https://github.com/ultralytics/ultralytics
8. Zhang, Y. et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. *ECCV*, 2022.
9. Lin, T.-Y. et al. Microsoft COCO: Common Objects in Context. *ECCV*, 2014.
10. Everingham, M. et al. The PASCAL Visual Object Classes (VOC) Challenge. *IJCV*, 88, 2010.
11. Johnson, A. et al. *MIMIC-IV Clinical Database Demo* (v2.2). PhysioNet, 2023. https://physionet.org/content/mimic-iv-demo/2.2/
12. Goldberger, A. et al. PhysioBank, PhysioToolkit, and PhysioNet. *Circulation*, 101(23), 2000.
13. Černek, A.; Sedmidubsky, J.; Budikova, P. REHAB24-6: Physical Therapy Dataset for Analyzing Pose Estimation Methods. *SISAP*, Springer, 2024.
14. Korfiatis, A. P. et al. PriMock57: A Dataset of Primary Care Mock Consultations. *ACL*, 2022.
15. Microsoft. *Text Analytics for health* e *Sentiment analysis*. Microsoft Learn, Azure AI Language.
16. Iglewicz, B.; Hoaglin, D. *How to Detect and Handle Outliers* (z-score modificado com MAD). ASQC, 1993.

17. Brasil. Lei nº 13.709/2018 (Lei Geral de Proteção de Dados Pessoais). ANVISA, RDC nº 657/2022 (software como dispositivo médico).
18. Boersma, P.; Weenink, D. *Praat: doing phonetics by computer.* https://www.praat.org
19. Jadoul, Y.; Thompson, B.; de Boer, B. Introducing Parselmouth: A Python interface to Praat. *Journal of Phonetics*, 71, 1-15, 2018. https://doi.org/10.1016/j.wocn.2018.07.001
20. Gemmeke, J. F. et al. Audio Set: An ontology and human-labeled dataset for audio events. *IEEE ICASSP*, 2017.
21. Plakal, M.; Ellis, D. *YAMNet.* TensorFlow Models, `research/audioset/yamnet`. https://github.com/tensorflow/models/tree/master/research/audioset/yamnet
22. Teixeira, J. P.; Oliveira, C.; Lopes, C. Vocal acoustic analysis: jitter, shimmer and HNR parameters. *Procedia Technology*, 9, 1112-1122, 2013.
23. Microsoft. *Fast transcription* (Azure AI Speech). Microsoft Learn.

Apêndice A. Módulo de áudio vocal (US10, US12, US14)

Detalha o módulo de áudio vocal resumido nas seções 3.2, 4 e 5.6: ingestão e pré-processamento (US10), biomarcadores vocais (US12) e classificação e geração de achados (US14). Toda saída do módulo é um *indicativo para avaliação clínica, não um diagnóstico*, e os limiares marcados `[EXP]` precisam de revisão por fonoaudiólogos e médicos antes de qualquer uso real.

A.1 Escopo e etapas

O módulo recebe a gravação da voz de um paciente (um áudio de voz do WhatsApp, ou uma consulta com médico e paciente), prepara o sinal, mede características da voz e publica achados no mesmo contrato `EventoAchado` dos demais detectores, de onde o motor de alertas (US19) os consome.

Etapa	O que faz	US
Ingestão e conversão	Lê WAV, FLAC, OGG/Opus, MP3, M4A, AAC, WMA, AMR e WebM. Grava WAV PCM 16 bits, 16 kHz, mono, relê e confere o arquivo	US10
Pré-processamento	Cópia com volume normalizado e silêncios longos removidos, usada só nas features. O original nunca é alterado	US10
Metadados e lote	Duração, taxa, canais, origem, licença, sha256 e status por áudio. Um arquivo ruim é registrado e o lote continua	US10
Biomarcadores vocais	F0 (média e variação), jitter local, shimmer local, HNR, intensidade, pausas, taxa de fala e de articulação, com Praat	US12
Eventos respiratórios	Tosse, respiração, sibilo, arfar e respiração ofegante, com YAMNet (AudioSet) e score por trecho	US12
Baseline do paciente	Compara o áudio com os anteriores do mesmo paciente e emite `desvio_baseline_vocal`	US12
Classificação e achados	`fadiga_vocal`, `dificuldade_respiratoria`, `possivel_disartria` ou normal (`sem_achados`), cada um com os critérios que o explicam	US14
Interfaces	CLI (`audio.ingestao.run`, `audio.vocal.run`), rota `POST /audio/vocal` e script de avaliação	US10, US12, US14

A transcrição (US11) e a análise de texto (US13) estão na seção 3.2 do corpo. Aqui a transcrição é *entrada opcional*: ela diz quais trechos são do paciente e quantas palavras ele falou.

A.2 Fluxo do áudio

```
áudio (qualquer formato aceito)
  │  audio.ingestao
  ├─ lê: soundfile/libsndfile (WAV, FLAC, OGG/Opus) ou, se falhar, ffmpeg (MP3, M4A, AAC, WMA, AMR,
WebM)
  ├─ convertido.wav ........ WAV PCM 16 bits, 16 kHz, mono (é o que vai ao Azure Speech)
  ├─ features.wav .......... cópia normalizada, sem silêncios longos (só para as features)
  └─ metadados.json ........ duração, taxa, canais, origem, licença, sha256, status
  │  audio.vocal
  ├─ segmentos do paciente: JSON do Azure Speech (US11) ou, sem ele, VAD por energia
  ├─ Praat/parselmouth: F0, jitter, shimmer, HNR, intensidade, pausas, taxa de fala
  ├─ YAMNet (ONNX): tosse, respiração, sibilo, arfar, com score por trecho
  ├─ baseline do próprio paciente (mediana/MAD dos áudios anteriores)
  ├─ regras: fadiga_vocal | dificuldade_respiratoria | possivel_disartria | normal
  └─ EventoAchado (modalidade audio) → barramento → motor de alertas (US19)
```

O fluxo tem sete passos:

1. **Entrada.** Um arquivo ou uma pasta de áudios, pela CLI ou por `POST /audio/vocal`. O `patient_id` é pseudonimizado antes de qualquer cálculo; sem a `PSEUDONYM_KEY` a execução falha cedo, antes de converter o áudio.
2. **Ingestão (US10).** Decodifica, valida, converte para WAV PCM 16 kHz mono e grava os metadados.
3. **Pré-processamento (US10).** Gera `features.wav` (normalizado, sem silêncios longos) e o mapa que leva cada instante do original ao instante equivalente da cópia.
4. **Segmentação.** Define os trechos de fala do paciente (transcrição ou VAD).
5. **Medição (US12).** Biomarcadores por segmento e eventos respiratórios no áudio todo.
6. **Classificação (US14).** Compara com a baseline do paciente, aplica as regras e escolhe a classe.
7. **Publicação.** Monta os `EventoAchado`, grava `eventos.jsonl` e `relatorio.json` e publica no barramento compartilhado. Severidade `alta` vira alerta sem mudar o motor.

Saídas por áudio, em `saida/audio/<audio_id>/`: `convertido.wav`, `features.wav`, `metadados.json`, `eventos.jsonl`, `relatorio.json` e, com `--transcrever`, `convertido.json` (a transcrição, com o paciente já pseudonimizado). O lote resume em `saida/audio/metadados.csv`, e a baseline de cada paciente fica em `saida/audio/baseline/<pt_...>.json`.

Qual sinal alimenta cada medida

A US10 manda normalizar o volume e remover silêncios longos **só na cópia usada para as features**. Isso decide o desenho: a remoção de silêncio apagaria justamente as pausas, e a normalização apagaria a diferença entre gravações altas e baixas. Por isso cada medida usa o sinal que não a distorce.

Medida	Sinal	Por quê
F0, jitter, shimmer, HNR	`features.wav` (recalculada em memória)	Não dependem do volume nem das pausas; a normalização ajuda o detector de F0

Medida	Sinal	Por quê
Proporção e duração das pausas, taxa de fala e de articulação	`convertido.wav`	A cópia perderia as pausas longas
Intensidade	`convertido.wav`	A normalização apagaria o nível real da gravação
Eventos respiratórios (YAMNet)	`convertido.wav`	Decisão de projeto: o ganho de até 5× da normalização amplificaria o ruído de fundo, e respiração é um som fraco que se confunde com ele

Os segmentos da transcrição estão no tempo do `convertido.wav`. `CopiaFeatures.indice` (`audio/ingestao/preparo.py`) mapeia cada amostra do original para a cópia, e `recortar(inicio_s, fim_s)` devolve o trecho equivalente.

A.3 Ingestão e conversão de formato (US10)

Pacote `audio/ingestao/`, parâmetros em `audio/ingestao/config.yaml` (versão **1.0.0**). Detalhes no relatório US10.

Item	Valor	Justificativa
Formato de saída	WAV, PCM 16 bits, mono, **16 kHz**	Formato recomendado pelo Azure Speech. Profundidade e canais são fixos no código; a taxa vem do YAML
Decodificação	`soundfile` (libsndfile) tentado primeiro; `ffmpeg` como plano B	Só WAV, FLAC e OGG/Opus (o formato dos áudios do WhatsApp) estão garantidos sem ffmpeg, o que permite testar sem ele. MP3, M4A, AAC, WMA, AMR e WebM dependem do ffmpeg
Reamostragem	`scipy.signal.resample_poly` (polifásica, razão reduzida pelo MDC das taxas)	Filtro anti-aliasing embutido; razão racional exata, como 160/441 para 44,1 → 16 kHz
Mono	média dos canais	O que o Azure Speech recomenda, ao custo de descartar o estéreo
Proteção contra estouro	se o pico passa de 1,0 após o downmix, o sinal é reduzido, não cortado	Evita distorção por *clipping*
Conferência	o WAV gravado é **relido** (`sf.info`) e validado: formato, codificação, taxa, canais e duração	Um arquivo fora do formato é apagado e vira erro, nunca segue adiante
Timeout do ffmpeg	300 s (`app/audio_convert.py`, parâmetro `timeout_s`)	Um arquivo malformado não trava o lote

Validação de entrada. Cada falha tem um erro tipado com código estável, usado em log, metadados e API:

Código	Quando
`arquivo_nao_encontrado`	caminho inexistente ou que não é arquivo
`formato_nao_suportado`	extensão fora de `wav, flac, ogg, oga, opus, mp3, m4a, aac, wma, amr, webm`, ou formato que exige ffmpeg ausente do PATH
`audio_vazio`	0 bytes, sem amostras ou silêncio digital (pico < 0,0001, ≈ −80 dBFS)
`audio_corrompido`	não decodifica, taxa inválida ou amostras NaN/Inf
`audio_curto`	duração < 0,5 s (não há fala para transcrever nem para medir jitter e shimmer)

Código	Quando
`audio_longo`	duração > 3600 s (a decodificação carrega o áudio inteiro como `float32`); conferido antes de decodificar
`audio_id_invalido`	`audio_id` que não serve de nome de pasta (ver A.14)

Lote. `ingerir_lote` registra cada falha em log e em `metadados.csv` (`status=erro`, com o código) e segue para o próximo arquivo. Um erro inesperado em um arquivo também não derruba o lote. A CLI sai com `0` (todos ingeridos), `1` (nada a processar ou configuração inválida) ou `2` (algum falhou).

Metadados por áudio (`metadados.json` e `metadados.csv`): `audio_id`, nome do arquivo (relativo à pasta de entrada, nunca o caminho absoluto), status e erro, sha256 e tamanho, formato, codec e decodificador de origem, duração, taxa e canais **do original**, `origem` e `licenca`, os mesmos dados do WAV convertido, da cópia de features (duração, silêncio removido, ganho aplicado), `original_preservado` (sha256 antes e depois), versão da configuração e instante do processamento. Pelo ffmpeg a taxa e os canais originais se perdem (o ffmpeg converte direto), e esses campos ficam vazios.

A.4 Pré-processamento e tratamento de ruído

A cópia de features (`audio/ingestao/preparo.py`) faz duas coisas, nesta ordem, e o `convertido.wav` e o original **não sofrem nenhuma delas**:

Etapa	Método	Parâmetros
Normalização de volume	o pico vai ao alvo, com ganho limitado	pico alvo **0,95**; ganho máximo **5×** (≈ 14 dB)
Remoção de silêncios longos	quadro é silêncio se o RMS fica abaixo do nível de fala da gravação; cada silêncio longo é encurtado	janela 25 ms; limiar **−30 dB** relativo ao percentil 95 dos RMS; só silêncios ≥ **1,0 s**; permanece **0,3 s** (metade de cada lado)

O limiar é **relativo ao nível de fala da própria gravação**, para valer em gravações altas e baixas. Nas pontas do arquivo fica só a metade vizinha à fala; no meio, metade de cada lado, o que preserva ataque e decaimento. Não se corta só o começo e o fim: cada silêncio de pelo menos 1 s é encurtado.

Como o módulo lida com ruído. O módulo **não aplica supressão de ruído** (filtro ou *denoising*). O que existe é um conjunto de salvaguardas para que o ruído não seja confundido com fala:

- O **ganho máximo de 5×** impede que a normalização amplifique uma gravação que é quase só ruído.
- O limiar de silêncio é relativo, mas há um **piso absoluto** do nível de fala em **−50 dBFS** (`pausas.nivel_minimo_fala_db`). Sem ele, um segmento só de silêncio ou de ruído de ambiente viraria "a fala de si mesmo". Esse caso foi achado por um teste: um segmento de silêncio digital era contado como fala.
- O **HNR** (razão harmônicos-ruído) mede o ruído *na voz* e entra como critério clínico (A.6).
- O **YAMNet** roda no `convertido.wav`, sem a normalização, pelo motivo da tabela da A.2.

Gravações com muito ruído de fundo reduzem o HNR e podem ser lidas como voz alterada (ver A.15).

A.5 Segmentação da fala do paciente

`audio/vocal/segmentos.py` decide *o que* medir. Há duas fontes:

- **Com a transcrição do Azure Speech (US11):** cada `fala` do paciente vira um segmento, com a contagem de palavras de que a taxa de fala precisa. `medico_falante` diz quem é o médico; **dois falantes sem essa informação são recusados**, porque errar mediria a voz do médico como a do paciente (mesma regra do `audio/text`, US13). A diarização é de até 2 falantes.
- **Exceção explícita:** áudio de **um só falante** (a gravação do próprio paciente, como um áudio de voz do WhatsApp) é tratado como paciente e marcado `papel_assumido`.
- A lacuna entre duas falas do paciente só conta como pausa dele se **não houver fala do médico no meio** (`contiguo_ao_anterior`); uma troca de turno não é pausa.
- **Sem transcrição:** VAD por energia (janela 20 ms; silêncios menores que 1 s ficam dentro do segmento) e **sem taxa de fala**. Os critérios que dependem dela ficam "não avaliáveis" e não contam.

Segmentos com menos de 0,3 s são ignorados (poucos ciclos de F0 para medir).

A.6 Biomarcadores vocais (US12)

Praat via `praat-parselmouth` **0.4.7** (Praat **6.1.38**), em `audio/vocal/biomarcadores.py`. Os parâmetros estão em `audio/vocal/config.yaml` (versão **1.0.0**), cada um marcado com a origem: `[MDVP]` (normalidade do MDVP, citada na US12), `[PRAAT]` (parâmetro padrão do Praat) ou `[EXP]` (**experimental**, sem validação clínica). Detalhes no relatório US12 e no desenho.

Medida	Método	Parâmetros
F0 (média e variação)	`To Pitch`; variação como desvio padrão em **semitons** em torno da média	faixa **75 a 500 Hz**, passo 10 ms `[PRAAT]`
Jitter local (%)	`To PointProcess (periodic, cc)` + `Get jitter (local)`	período 0,1 ms a 20 ms; razão máxima entre períodos 1,3 `[PRAAT]`
Shimmer local (%)	`Get shimmer (local)` sobre o mesmo `PointProcess`	razão máxima entre amplitudes 1,6 `[PRAAT]`
HNR (dB)	`To Harmonicity (cc)`, média	limiar de silêncio 0,1; 1,0 período por janela `[PRAAT]`
Intensidade (dB)	`To Intensity`, média dos quadros de fala (≥ máximo − 25 dB)	janela mínima do Praat (6,4 / piso de F0)
Pausas	silêncio de pelo menos **0,25 s**, a **25 dB** abaixo do nível de fala do paciente (percentil 95 dos RMS)	piso absoluto de −50 dBFS `[EXP]`
Taxa de fala	palavras da transcrição ÷ duração da fala, **incluindo** pausas	só com transcrição
Taxa de articulação	palavras ÷ (duração da fala − pausas)	só com transcrição

Voz sustentada mínima. Com menos de **0,25 s** de voz sustentada em um segmento não há períodos suficientes, e jitter, shimmer e HNR ficam **nulos**. Medidas que o Praat não consegue calcular viram `None`, **nunca NaN** (o contrato rejeita NaN).

Agregação por áudio. Os tempos são somados. Jitter, shimmer e HNR são médias ponderadas pela voz sustentada de cada segmento, a intensidade é ponderada pela duração, e o F0 junta os quadros vozeados de todos os segmentos.

Verificação do método. Os testes usam voz sintética de parâmetros conhecidos (`audio/vocal/injecao.py`: pulsos glóticos com 20 harmônicos, jitter, shimmer e SNR controlados): a F0 foi medida em **148,8 Hz** contra **148,35 Hz** esperados (0,3 %), e um jitter injetado de **2 %** foi medido em **1,96 %**.

A.7 Eventos respiratórios com YAMNet (US12)

`audio/vocal/respiratorio.py`. O YAMNet é uma rede MobileNet treinada no AudioSet que devolve **521 scores** (uma classe cada) a cada **0,48 s**, para uma janela de **0,975 s**. Roda em **ONNX** (`onnxruntime`, CPU), sem TensorFlow, o que o mantém leve e compatível com o Python do projeto. O modelo e o mapa de classes são baixados por `scripts/baixar_modelos_audio.py` para `models/audio/` (fora do Git), com o **SHA-256 fixado**: a URL aponta para uma ramificação mutável, então é o hash que garante que o arquivo é o validado.

Classe do AudioSet	Tipo do evento	Severidade
Cough	`tosse`	baixa
Breathing	`respiracao`	baixa
Wheeze	`sibilo`	média
Gasp	`arfar`	média
Pant	`respiracao_ofegante`	média

Um **evento** é um trecho contínuo de quadros com o score da classe acima do mínimo, e leva o score máximo e o médio.

Parâmetro	Valor	Origem
Score mínimo	**0,30**	`[EXP]`
Lacuna tolerada dentro de um trecho	1 quadro	
Máximo de eventos por áudio	20 (ficam os de maior score)	evita inundar o painel com uma respiração contínua
Fala do médico	quadros cujo centro cai em fala do médico são descartados	só com transcrição

A tabela de classes do YAML é **conferida contra o mapa de 521 nomes** no carregamento. Sem o modelo baixado, a análise segue sem o critério respiratório e o relatório registra `respiratorio.disponivel = false`, em vez de falhar ou de fingir que não houve evento.

A.8 Baseline do paciente (US12)

`audio/vocal/baseline.py`. Com pelo menos **3 áudios anteriores** do mesmo paciente (pelo pseudônimo), cada métrica é comparada com a mediana do histórico por um **z robusto**, o mesmo critério usado nos sinais vitais (US15):

```
z = (x − mediana) / max(1,4826 · MAD, piso da métrica)
```

Um desvio com `|z| ≥ 3,5 [EXP]` gera `desvio_baseline_vocal`, de severidade baixa (média com 3 ou mais métricas fora). O **piso** da dispersão evita um z enorme quando o histórico é quase constante:

Métrica	Piso	Métrica	Piso
`jitter_local_pct`	0,10	`intensidade_db`	2,0

Métrica	Piso	Métrica	Piso
`shimmer_local_pct`	0,50	`proporcao_pausas`	0,05
`hnr_db`	1,0	`taxa_fala_pps`	0,2
`f0_media_hz`	5,0	`taxa_articulacao_pps`	0,2
`f0_desvio_st`	0,5		

O histórico guarda no máximo 30 áudios, em um JSON por paciente indexado **só pelo pseudônimo**, gravado de forma atômica. Um áudio que disparou uma classe **não entra** na baseline, para que uma alteração não vire o novo normal. A baseline também alimenta o critério `intensidade_caiu` (queda de 6 dB em relação à mediana do paciente).

A.9 Classificação (US14)

`audio/vocal/classificacao.py`. Cada critério é um predicado sobre os biomarcadores, com limiar vindo do YAML:

Critério	Satisfeito quando	Origem
`jitter_alto`	jitter local > **1,04 %**	`[MDVP]`
`shimmer_alto`	shimmer local > **3,81 %**	`[MDVP]`
`hnr_baixo`	HNR < **20 dB**	`[MDVP]`
`pausas_altas`	pausas ocupam > **40 %** da fala	`[EXP]`
`fala_lenta`	taxa de fala < **1,8** palavras/s (≈ 108 por minuto)	`[EXP]`
`articulacao_lenta`	taxa de articulação < **2,2** palavras/s	`[EXP]`
`f0_monotona`	desvio de F0 < **2,0** semitons	`[EXP]`
`intensidade_caiu`	intensidade ≥ **6 dB** abaixo da baseline do paciente	`[EXP]`
`eventos_respiratorios`	≥ **2** eventos, ou um com score ≥ **0,60**	`[EXP]`

Valores exatamente no limiar não contam como alteração. Um critério **sem dado** (por exemplo, taxa de fala sem transcrição, ou intensidade sem baseline) fica "não avaliável" e **não conta a favor**.

Uma classe dispara com `minimo` critérios satisfeitos e, quando há, pelo menos um de `exigir_um_de`:

Classe (`event_type`)	Critérios	Mínimo	Exige um de	Severidade
`dificuldade_respiratoria`	eventos respiratórios, pausas altas, fala lenta, HNR baixo	2	eventos respiratórios ou pausas altas	média; **alta com 3**
`fadiga_vocal`	intensidade caiu, F0 monótona, pausas altas, fala lenta	2	n/a	média
`possivel_disartria`	jitter, shimmer, HNR, articulação lenta, pausas altas	3	articulação lenta ou pausas altas	média
normal	nenhuma classe disparou (e nenhum outro achado)	n/a	n/a	`sem_achados` (info)

Uma classe pode disparar junto com outra: **cada uma vira um evento**, com os critérios que a explicam. O `score` do evento é a fração de critérios satisfeitos (3 de 4 = 0,75). A única saída de severidade **alta**, a de maior prioridade no motor de alertas, é `dificuldade_respiratoria` com 3 critérios; as outras classes param em severidade média, por desenho.

Explicabilidade. Cada achado de classe traz o `summary` com cada critério satisfeito (valor e limiar), as `features` com as medições e as `regras_acionadas`, os `thresholds` que a classe usa mais a versão da configuração, os `models` (Praat, parselmouth, YAMNet, Azure Speech) e o artefato `audio_segment`. O `summary` sempre começa com "Indicativo para avaliação clínica, não é diagnóstico", e `features.aviso` repete a frase, porque o e-mail de alerta lista as features.

A.10 Contrato e alertas

Os seis `event_type` de áudio já existiam no catálogo do contrato; **nada mudou no catálogo nem no schema**.

Achado	`EventoAchado`
classe disparada (US14)	`fadiga_vocal`, `dificuldade_respiratoria` ou `possivel_disartria`
trecho do YAMNet (US12)	`evento_respiratorio`, com o `score` do trecho
métricas fora da baseline (US12)	`desvio_baseline_vocal`
nenhuma classe e nada mais	`sem_achados` (info): é a classe "normal"

Campos comuns: `patient_id` pseudonimizado; `timestamp` = `recorded_at` + posição do trecho no áudio; `window` com início e fim; `model_version = audio-achados@0.1.0`; `context.source_id = audio_id`; URIs de artefato **relativas**. Os valores numpy viram `float`/`int` nativos e NaN vira `None`.

A publicação vai ao **barramento compartilhado** (`app.barramento.get_barramento()`, importado no nível do módulo para ser substituível nos testes). Um evento de severidade alta vira alerta sem mudar o motor, com prioridade `alta` e origem `padrao:alta`.

Não há regra de áudio em `config/alertas_regras.yaml`, de propósito. Uma regra por tipo *substituiria* a prioridade e anularia a lógica de "2 critérios = média, 3 = alta".

A.11 Interfaces

Interface	Uso
`python -m audio.ingestao.run --input <pasta> --origem ... --licenca ...`	só a US10: converte uma pasta, um arquivo ruim não interrompe o lote
`python -m audio.vocal.run --audio <wav> --patient-id <id> --recorded-at <ISO UTC> [--transcrever --medico-falante N]`	US10 + US12 + US14 de ponta a ponta; liga o motor de alertas. Saída `0`, `1` (configuração) ou `2` (algum áudio falhou)
`POST /audio/vocal` (multipart)	o mesmo pela API; `?detalhes`, `?publicar` e `?atualizar_baseline`
`python scripts/baixar_modelos_audio.py`	baixa o YAMNet em ONNX, com SHA-256 conferido
`python scripts/avaliar_us14.py`	avaliação em casos injetados (A.12)

A rota valida tudo o que não depende do áudio (formato, metadados, configuração, chave de pseudonimização, credenciais do Speech) **antes** de gravar o upload e antes de gastar uma chamada ao Speech. Respostas de erro: `400` formato não suportado, `409` `audio_id` já usado, `413` acima do limite de tamanho, `422` áudio ou análise inválidos, `503` configuração ou chave ausente. O áudio enviado é preservado em `saida/audio/<audio_id>/original.<ext>`.

A.12 Resultados detalhados

Os números abaixo vêm de testes automatizados, da avaliação em casos injetados (`saida/audio/avaliacao_us14.json`) e de uma rodada com áudios reais. Cada linha diz de qual das três fontes veio.

US	Métrica	Meta	Obtido	Atingida?	Observação
US10	Entradas convertidas para WAV PCM 16 kHz, 16 bits, mono	todas as válidas	cada conversão é relida e validada. Testado de 8 a 48 kHz, mono e estéreo, OGG/Vorbis e OGG/Opus, sem ffmpeg	✔	`tests/audio/test_ingestao_conversao.py`
US10	Áudio ruim não interrompe o lote	sim	erro tipado em log e em `metadados.csv`; a CLI sai com 2	✔	`test_ingestao_lote.py`, `test_ingestao_validacao.py`
US10	Normalização e corte de silêncio só na cópia; original preservado	sha256 igual	`original_preservado` por áudio; `convertido.wav` mantém pausas e volume	✔	`test_ingestao_preparo.py`
US10	Nenhum áudio versionado	0	0 (teste roda `git ls-files`)	✔	
US12	Features por segmento do paciente	F0, jitter, shimmer, HNR, intensidade, taxa de fala, pausas	todas implementadas	✔	taxa de fala exige transcrição
US12	F0 em voz sintética	n/a	148,8 Hz medido contra 148,35 Hz esperado (0,3 %)	n/a	`test_vocal_biomarcadores.py`
US12	Jitter injetado de 2 %	n/a	1,96 % medido	n/a	
US12	Limiares MDVP documentados	jitter 1,04 %, shimmer 3,81 %, HNR 20 dB	no YAML, marcados `[MDVP]`	✔	com ressalva: áudio comprimido (A.13)
US12	Eventos respiratórios com modelo do AudioSet e score por segmento	sim	YAMNet real roda: silêncio gera `Silence` = 1,0; tom de 440 Hz gera `Telephone/Dial tone`; a conta de quadros fecha	✔	`test_vocal_respiratorio.py`
US12	YAMNet em tosse ou respiração **reais**	sem meta	não medido	✘	não há gravações rotuladas no projeto
US12	Baseline do próprio paciente	≥ 3 áudios	implementada, com z robusto e piso por métrica	✔	`test_vocal_baseline.py`
US14	Recall por classe (casos injetados)	≥ 0,80	fadiga **1,00** · dificuldade respiratória **1,00** · disartria **1,00** (IC95 0,94 a 1,00; 60 casos cada)	✔	**avaliação circular** (ver abaixo)
US14	Voz normal sintética sem nenhuma classe disparada	n/a	**60/60**; falso positivo 0/60 (0 %)	n/a	
US14	Achado de severidade alta chega ao motor de alertas	sim	`dificuldade_respiratoria` com 3 critérios gera alerta `padrao:alta`	✔	`test_vocal_analise.py`
US14	Ao menos 1 exemplo de alteração vocal detectada	1	1 caso **sintético** (seção 5.6)	✔ (sintético)	os 3 áudios reais não dispararam classe

US	Métrica	Meta	Obtido	Atingida?	Observação
Todos	Testes automatizados do módulo	n/a	227 funções de teste em 16 arquivos	n/a	`tests/audio/`; sem rede e sem pesos (o YAMNet real é opcional)

Avaliação em casos injetados (US14)

`scripts/avaliar_us14.py`: 3 sementes × 20 casos por classe (**60 por classe**), voz sintética passando pelo pipeline inteiro (ingestão, Praat, regras). Um caso é acerto quando a classe do rótulo dispara (outras podem disparar junto); em "normal", quando nenhuma dispara. Meta de recall: 0,80.

Rótulo	Recall (IC 95 %)	Classes que dispararam
normal	1,00 [0,94; 1,00]	nenhuma em 60/60
`fadiga_vocal`	1,00 [0,94; 1,00]	fadiga 60, dificuldade respiratória 26
`dificuldade_respiratoria`	1,00 [0,94; 1,00]	dificuldade 60, fadiga 58, disartria 52
`possivel_disartria`	1,00 [0,94; 1,00]	disartria 60, fadiga 25

As faixas de parâmetros de cada rótulo (jitter, shimmer, SNR, variação de F0, ritmo de fala, pausas) descrevem a alteração pretendida e **não foram ajustadas até as regras acertarem**. Mesmo assim:

- **A avaliação é circular.** Os limiares experimentais e os parâmetros injetados são dos mesmos autores, e os casos foram injetados bem acima dos limiares. O recall de 100 % mostra que as regras e a implementação respondem às alterações que procuram, **não** que o sistema acerta em pacientes. Falta voz real rotulada por fonoaudiólogo.
- **O YAMNet não é avaliado.** Os eventos respiratórios entram como scores injetados (`sessao_sintetica`), então só a lógica das regras é testada.
- **As classes se sobrepõem.** Fadiga e dificuldade respiratória compartilham fala lenta e pausada, e voz soprosa parece disártrica na acústica. É a leitura mais útil da tabela.

Rodada com áudio real

Três áudios de voz (OGG/Opus, 5,6 a 8,4 s, 16 kHz mono) rodaram pela CLI, sem rede e sem transcrição (portanto sem taxa de fala):

Amostra	F0	Jitter local	Shimmer local	HNR	Pausas
1	105 Hz	2,31 %	14,8 %	12,7 dB	16 %
2	94 Hz	2,02 %	12,1 %	14,0 dB	7 %
3	97 Hz	1,97 %	10,9 %	13,7 dB	30 %

Os três ultrapassam os três limiares MDVP ao mesmo tempo e saíram `normal` apenas porque a disartria exige fala lenta ou pausas, e sem transcrição isso não é avaliável. Três amostras com o mesmo padrão indicam **artefato**, não três alterações (A.13).

Metas não atingidas e por quê

- **Validação clínica dos limiares.** Os limiares `[EXP]` (pausas, taxa de fala e de articulação, variação de F0, queda de intensidade, score respiratório) não têm validação. Dependem de gravações rotuladas por fonoaudiólogo, que o projeto não tem.

- **Exemplo de alteração em áudio real.** Nenhum dos 3 áudios reais disparou uma classe, então o exemplo documentado é sintético. **Para o vídeo de demonstração falta uma gravação real alterada.**
- **YAMNet em sons reais.** Foi verificado em silêncio e em tons, não em tosse ou respiração reais. O score mínimo de 0,30 é experimental, e o TensorFlow avisa que os scores do YAMNet não são probabilidades calibradas entre classes.
- **Transcrição real com o Azure Speech.** O caminho com `--transcrever` não foi exercitado contra o serviço real nesta entrega: a assinatura Azure do grupo está desativada (ver a seção 4). Os testes usam uma transcrição sintética no formato da US11 e um transcritor substituível.
- **Alertas.** Na rodada do exemplo da seção 5.6, o alerta foi registrado no painel mas ficou `failed` ao enviar, porque nenhum canal (SMTP ou webhook) estava configurado no ambiente.

A.13 Áudio real e um defeito encontrado

Áudio real do WhatsApp: três amostras fora dos limiares, classificadas como normais

Os três áudios reais (OGG/Opus) ultrapassaram **jitter, shimmer e HNR ao mesmo tempo** (jitter ≈ 2 %, shimmer de 11 a 15 %, HNR de 13 a 14 dB). O padrão idêntico nas três amostras aponta para artefato: codec com perdas, gravação de celular e fala encadeada, quando os limiares MDVP são de gravação de qualidade clínica, em geral de vogal sustentada. O sistema reagiu de três formas:

- só saíram `normal` porque a disartria exige também fala lenta ou pausas;
- o achado leva `audio_comprimido = true` (formatos com perdas: `ogg, oga, opus, mp3, m4a, aac, wma, amr, webm`) e o relatório, uma nota de que jitter, shimmer e HNR podem estar superestimados;
- a conclusão registrada é que **estes limiares não devem ser aplicados a áudio de WhatsApp sem calibração**.

Um defeito que a avaliação encontrou

Na primeira avaliação com casos injetados, **60 de 60 casos de disartria também disparavam "dificuldade respiratória"**: voz alterada (HNR baixo) mais fala lenta bastava, sem nenhum sinal respiratório. A regra passou a exigir um evento respiratório (YAMNet) ou pausas frequentes (fala entrecortada para respirar), o parâmetro `exigir_um_de`, e a sobreposição caiu a 0/60 sem perder recall. Como a medição seguinte usa os mesmos dados, essa melhora também é circular. O motivo da regra é clínico, não o número: dificuldade respiratória precisa de sinal respiratório ou de fala entrecortada.

A.14 Dados, licenças e LGPD do áudio

Dados e componentes

Nenhum áudio é versionado. Os testes geram voz sintética em `tmp_path`, e o `.gitignore` cobre `tratamento_audio/` e as extensões `*.wav *.ogg *.oga *.opus *.m4a *.mp3 *.flac *.aac *.wma *.amr`. Não há conjunto público de voz patológica rotulada que o projeto possa versionar.

Fonte	Uso	Licença
Voz sintética (`audio/vocal/injecao.py`)	testes e avaliação da US14	gerada pelo grupo; nada a redistribuir

Fonte	Uso	Licença
3 áudios de voz (OGG/Opus)	rodada com áudio real	uso local, não versionados. O consentimento do titular precisa ser confirmado (o texto padrão de proveniência do módulo diz isso)
YAMNet em ONNX, espelho de terceiros `andrelgomes/yamnet-onnx` (conversão do modelo do Google)	eventos respiratórios	Apache-2.0, conforme o cabeçalho do script de download. SHA-256 fixado; é uma conversão de terceiros, e a alternativa é converter do TF Hub com `tf2onnx`
`yamnet_class_map.csv` (`tensorflow/models`)	nomes das 521 classes	Apache-2.0
Praat, via `praat-parselmouth` 0.4.7	biomarcadores	**GPL-3.0 ou posterior** (parselmouth; o Praat é GPL-2.0 ou posterior). A GPL vale para quem redistribuir o software: avaliar antes de qualquer distribuição
`soundfile` (libsndfile), `scipy`, `onnxruntime`	leitura, reamostragem e inferência	licenças permissivas (BSD, MIT) e LGPL no libsndfile; conferir antes de redistribuir

Tratamento LGPD

A voz de um paciente é dado pessoal sensível (LGPD, art. 5º, II). As medidas do módulo:

Medida	Como
Pseudonimização na borda (art. 13 §4º)	`patient_id` vira `pt_<24 hex>` por HMAC-SHA256 com `PSEUDONYM_KEY` (`contratos/pseudonimizacao.py`) antes de qualquer cálculo. O ID de origem nunca vai para o disco nem para a resposta: o JSON da transcrição é gravado com o pseudônimo, e a baseline é indexada só por ele (um nome de arquivo fora do padrão é recusado)
Nada de áudio no Git	`.gitignore` e um teste que roda `git ls-files` e falha se achar uma extensão de áudio
Minimização e evidência por referência	o evento guarda a URI relativa do trecho (`audio_segment`), não o conteúdo; os metadados guardam o nome do arquivo (relativo à pasta de entrada) e nunca o caminho absoluto
Proveniência	cada áudio registra `origem` e `licenca` (`--origem`, `--licenca`); o padrão avisa que o consentimento (LGPD) do titular deve ser confirmado antes do uso
Processamento local	Praat, YAMNet e as regras rodam na máquina. Só a transcrição **opcional** (`--transcrever`) envia o áudio ao Azure Speech; região e termos na seção 6
Segurança do `audio_id`	ele vira nome de pasta. O padrão do contrato aceitaria `..`, que sairia da pasta de saída (e o tratamento de erro apagaria um diretório pai). Exige-se que comece por letra ou número, e um `audio_id` repetido responde `409` em vez de sobrescrever o original preservado
Limite de upload	a API recusa upload acima do limite configurado (`AUDIO_MAX_UPLOAD_MB`, `413`) e apaga o que gravou
Auditoria	`eventos.jsonl` e o barramento são JSONL de acréscimo; `original_preservado` e o sha256 provam que o original não mudou

A.15 Limitações e trabalhos futuros do áudio

Limitações

- **Áudio comprimido e fala encadeada.** Os limiares MDVP são de gravação de qualidade clínica. Em formatos com perdas (Opus do WhatsApp, MP3), os três limiares estouram juntos (A.13). O achado leva `audio_comprimido`, mas o sistema **não corrige** o efeito.
- **Sem supressão de ruído.** O módulo só se protege do ruído (ganho máximo, piso de nível, HNR; A.4). Ruído de fundo reduz o HNR e pode ser lido como voz alterada. É o comportamento esperado da medida; não foi quantificado.
- **Limiares `[EXP]` sem validação.** E a origem primária dos limiares `[MDVP]` (1,04 %, 3,81 %, 20 dB) vem do texto da US12; este relatório não a conferiu em uma fonte primária.

- **Avaliação circular e sintética.** A US14 mede a lógica das regras em voz sintética com alterações injetadas por quem escolheu os limiares. Não mede validade clínica nem o YAMNet.
- **Sobreposição de classes.** Fala lenta e pausada é comum a fadiga e dificuldade respiratória, e voz soprosa parece disártrica na acústica.
- **Intensidade** é relativa ao ganho do microfone e só serve entre gravações em condições parecidas.
- **YAMNet.** Verificado em silêncio e em tons, não em sons respiratórios reais. As classes do AudioSet são genéricas: o modelo não sabe de quem é a tosse, e só descarta a fala do médico quando há transcrição com diarização. Os scores não são probabilidades, e o modelo ONNX é uma conversão de terceiros.
- **Sem transcrição, sem taxa de fala.** Os critérios `fala_lenta` e `articulacao_lenta` ficam não avaliáveis, e a disartria, que exige 3 critérios incluindo um de articulação ou pausas, quase não dispara. Já a taxa de fala em palavras por segundo depende do idioma e do tipo de fala.
- **Papel assumido.** Áudio de um só falante é tratado como paciente sem verificação. A diarização é de até 2 falantes.
- **Baseline com poucos áudios.** O mínimo é 3, e a baseline de um paciente ainda pode ser formada por gravações em condições diferentes. A trava do arquivo é por processo, então a API deve rodar com um único worker.
- **Processamento síncrono.** A rota processa o áudio dentro da requisição. Não há tempo real nem *streaming* de áudio.
- **Vários `--audio` na CLI** compartilham o mesmo `--recorded-at`; gravações em horários diferentes pedem uma execução por gravação.

Trabalhos futuros

1. **Calibração e validação clínica** dos limiares com gravações rotuladas por fonoaudiólogo, separando por tipo de tarefa (vogal sustentada ou fala encadeada) e por codec.
2. **Avaliar o YAMNet em tosse e respiração reais** e, se preciso, treinar um classificador dedicado a sons respiratórios.
3. **Estimativa de SNR e supressão de ruído** como etapa opcional, com um portão de qualidade que avise quando a gravação não serve para medir jitter, shimmer e HNR.
4. **Rodar a transcrição real** (Azure Speech) quando a assinatura for reativada, e medir o efeito da taxa de fala nas classes.
5. **Biomarcadores adicionais** (openSMILE, conjunto eGeMAPS) para comparar com o Praat.
6. **Fusão multimodal (US18):** usar os eventos de áudio com sinais vitais e vídeo em `risco_multimodal`.
7. **Baseline em banco de dados** e processamento assíncrono para suportar vários *workers* e gravações longas.
8. **Estudo retrospectivo** com eventos reais rotulados e revisão dos limiares e prioridades pela equipe clínica.